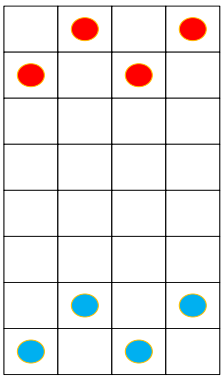
תכנות ברשת Net.
עבודת גמר סמסטר (פרויקט) .

תיאור כללי

נגדיר משחק לשני שחקנים, שדומה למשחק הדמקה, באופן הבא:
לוח המשחק הוא בגודל של 8 על 4, ונראה כך:



משחקים באופן דומה למשחק הדמקה המקורי (קיימות מספר גרסאות), אך לצורך תרגיל זה מוגדרים החוקים הבאים:
הליכה רגילה קדימה מתבצעת באלכסון בלבד, משבצת אחת, ובתנאי שהמשבצת פנויה.
תוספת לחוקי הדמקה: כל חייל רשאי לבצע פעם **אחת בלבד במהלך המשחק** צעד של משבצת אחת באלכסון אחורה, בתנאי שהמשבצת פנויה. **במהלך צעד זה לא ניתן לבצע אכילה**.
חייל יכול לאכול באלכסון קדימה בלבד, בקפיצה מעל חייל יריב אחד, ובתנאי שהמשבצת שמעבר לו פנויה. בכל תור מותר לבצע אכילה של **חייל אחד בלבד**. (כלומר רק 'קפיצה' אחת מעל נאכל אחד)
כל צד חייב לבצע מהלך כאשר הגיע תורו.
מטרת המשחק היא להגיע ראשון לשורה הראשונה בצד של היריב, כאשר די בהגעה של חייל אחד. בנוסף, שחקן מנצח גם אם הצליח לחסום את היריב כך שאין לו אף מהלך חוקי אפשרי בתורו.

מכיוון שהמשחק מתנהל מול ה־Server, רק לשחקן האנושי קיימת מגבלת זמן לכל צעד. ברירת המחדל היא 10 שניות, וניתן לבחור באמצעות Combo אחת מהאפשרויות: 2, 5 או 15 שניות.
אם השחקן האנושי אינו עומד במגבלת הזמן, הוא מפסיד את המשחק באופן אוטומטי.

נשחק את המשחק ברשת.
כל שחקן (לקוח) משחק כנגד ה- *Server*, כאשר ה- *Server* עונה לכל שחקן על צעד המשחק שלו בצעד נגדי.
ה- *Server* מרכז בנוסף גם את בסיס הנתונים ואת אתר האינטרנט של המשחק.
אין קשר בין המשחקים של הלקוחות השונים, וכולם יכולים לשחק מול ה- *Server* במקביל.

עבור ההגשה / ההגנה, יש להציג בפרויקט 4 תיקיות שונות ונפרדות לפי הפירוט הבא:

1. תיקיית לקוח אחד לדוגמא, שמשחק את המשחק.
2. תיקיית פרויקט של *ASP.NET Core Web App (...Razor Pages content)*, ובו יהיה אתר האינטרנט של המשחק.
באתר זה נבצע פעולות כגון תפעול בסיס הנתונים, רישום שחקנים למשחקים, הפעלת שאילתות וקבלת תוצאות,
וכך הלאה. בנוסף, דפי ה- *Server* של ה- *Razor* ישגרו לכל שחקן את צעד התשובה של ה- *Server*.
3. תיקייה ובה קובץ *dacpac* אחד עבור כל בסיס נתונים שנוצר כחלק מפרויקט ה- *Razor* ו/או *Web API*.
יש להקפיד על כך ש- שם הקובץ יהיה בדיוק כמו השם של בסיס הנתונים במקור.
יש לבחור באפשרות של *Extract schema and data* כדי לשמור גם את הנתונים.
בנוסף, תיקיה זו תכיל עבור כל טבלה קיימת של בסיס נתונים את קובץ ה- *Sql* שלו.
(יש לשמור ממצב ה- *design* של כל טבלה.)
4. תיקייה ובה קובץ ה- **Known Issues**:
בקובץ זה אתם מתעדים את הבעיות שידועות לכם בתרגיל ולא הספקתם לטפל בהן או שאינכם יודעים כיצד:
באגים, פיצ'רים שעדיין לא טיפלתם בהם, מספרי הסעיפים שלא היה זמן לממש אותם וכו'.
יתכן בהחלט שההגשה שלכם טובה אבל לא מושלמת. תיעוד של ה *Known Issues* - הינו כלי מקובל בתעשייה,
וזאת על-מנת לתעד את הנקודות שהפיתוח מודע אליהן.
ההתייחסות לבעיות שתיעדתם בקובץ ה *Known Issues* - תהיה סלחנית יותר לעומת בעיות שיימצאו בבדיקה ולא תועדו.
בנוסף, יש לרשום בקובץ זה את פרטי המגישים.
שימו לב: חובה להגיש קובץ *Known Issues* גם אם לא קיימים *Known Issues* .
5. **לסיכום: סה"כ יש להגיש הן באתר והן עבור ההגנה את 4 התיקיות הנ"ל בתוך קובץ *zip* אחד.**

בסיס הנתונים

6. בפרויקט בסיס נתונים מרכזי שנמצא בפרויקט ה- *Razor*.
7. לקוח יחזיק בסיס נתונים אחר רק כדי לשחזר משחקים (ראה בהמשך).
את שאר המידע הוא יקבל ע"י פניה ל- *Server*.
8. בסיסי הנתונים יכללו טבלאות בהתאם, לפי מה שצריך, כולל קשרי גומלין וכיוצ"ב.

Web API Core

9. התקשורת השוטפת בין כל לקוח לבין השרת תבוצע בעזרת *Web Api Core* כפי שנלמד בכיתה.

אתר האינטרנט

10. האתר יכיל:
- דף הרשמה:
האתר יציג טופס הרשמה עבור שחקנים הנרשמים למשחק.
בתחילת הדף יוצג למשתמש שדה לבחירת מספר השחקנים המשתתפים באותו משחק (עד 10 שחקנים).

כל המשתתפים במשחק יושבים יחד מול אותו מסך ומשחקים בתורות, ולכן נחשבים ל*Client* אחד לעניין המשחק עצמו. עם זאת, כל אחד מהמשתתפים נרשם בנפרד בבסיס הנתונים כשחקן המשויך לאותו משחק.

- בעזרת פקודות *Server* מתאימות, כמו שעשינו בכיתה, יש לייצר טופס שמכיל שורות הרשמה בכמות זאת, כאשר בכל שורה יהיו פקדים לקליטת הנתונים השונים של כל שחקן. יש לבצע כפי שעשינו בכיתה, לעבוד עם *[BindProperty]* בהתאם וכו'.

בכל שורה: יש לעבוד עם מספר שדות שונים מסוגים שונים, כמקובל, כולל שדות חובה.

שדות החובה יכילו לפחות: שם פרטי: לפחות *2* אותיות, מספר מזהה: כל אחד יכול לבחור עבור עצמו מספר מזהה, מספר שלם בין 1 ל- 1000, אולם התוכנית לא תאפשר את בחירתו באם כבר קיים מספר כזה במערכת, כלומר בבסיס הנתונים שנמצא ב- *Server*.

טלפון: בדיוק *10* ספרות.

מדינה: תהיה אפשרות לבחור מ- *Combo* מתוך מספר אפשרויות. רשימת המדינות תגיע מ- ה- *Server*, ושינוי שלה ב- *Server* יופיע ב- *Combo*.

יש לעבוד עם *div, class,...* וכו' בעיצוב הטפסים, כלומר טפסים מעוצבים לפי מה שעשינו בכיתה עם שימוש ב- *bootstrap* קיימים וכו'. בנוסף, לבצע עם *BindProperty* מתאים שיקושר לתיבות הקלט בהתאם.

יש לבצע *Validtion* עם *asp-validation-for* כדי לוודא שיש ערכים בשדות החובה, ושמתקיימים שאר האילוצים הנ"ל. יש גם להציג *asp-validation-summary* בהתאם.

באם *Validtion* נכשל יש להציג הודעה מקורית משלך, מותאמת לכל שדה.

(ולא את הודעת ברירת המחדל של *Razor*)

משחק חדש מאופשר מיד רק עם סיום מלא של ההרשמה באופן תקין ב- *Server*, וקבלת אישור ממנו.

- דף/דפים נוספים בהתאם, בהם נציג תוצאות של שאילתות מבוקשות, כפי שיפורט בהמשך.
- דף 'אודות' ובו פרטי המגישים.
- יש לאפשר *link*-ים מתאימים שיאפשרו מעבר בין הדפים השונים. יש לעבוד רק עם *asp-page* לצורך יצירת הקישורים.

לקוח

11. עבור כל לקוח נציג את לוח המשחק, כמתואר למטה כאן בהמשך, ובנוסף חלוניות אחרות בהתאם לצורך.

לוח המשחק

12. נממש את הלוח אצל כל לקוח בעזרת אפליקציה 'שולחנית', *winform* במקרה שלנו.

מי שיודע *wpf* אמנם לא למדנו, אבל אפשר.

(הסעיפים הבאים מתייחסים ל- *winform*, ומי שעושה עם *wpf* יממש בהתאמה את אותן דרישות.)

13. יהיה ניתן לבחור את הזמן הקצוב לשחקן/ים האנושיים, כפי שתואר קודם לכן. יש להציג כמה זמן נשאר בכל תור.
14. למימוש הלוגיקה של לוח המשחק יש לעבוד **רק עם מטריצות מהסוג *rectangle***, כפי שלמדנו בכיתה.

15. **<u>אילוץ</u>**: יש לשלב אנימציה יצירתית של הצורות ו/או של המשבצות בהן מתבצעת פעילות. חובה לשם כך לשלב עבודה עם המחלקה של *Graphics* ועם אנימציה, כלומר שילוב של *Timer* ו/או דברים שמשתנים עם הזמן, למשל תנועה ע"פ זמן, שינוי צורות ע"פ זמן, שינוי צבע מדורג של המשבצת ע"פ הזמן וכיו"ב.
יש לאפשר ציור חופשי עם העכבר על לוח המשחק. למשל, אם אני רוצה להסביר על מהלכים שבוצעו, אני יכול להקיף בעיגול (ביד חופשית) משבצות מסוימות, לשרבט חצים וכו'. גם כאן יש לעבוד עם המחלקה של *Graphics*.
חובה לעבוד עם *Bitmap* בהתאם לצורך, כפי שלמדנו בכיתה. (כלומר אסור *List* או משהו ששומר נקודות שבהן ציירנו קודם לכן). בנוסף, תהיה כמובן האפשרות לנקות את "הציורים של ההסברים".
16. כאשר אחד הצדדים מנצח, כל הכלים שלו מהבהבים בירוק, הלוך חזור במשך כ- *5* שניות! (לא שינוי צבע חד פעמי אלא הבהוב.)

<u>אפשרויות משחק</u>

17. התוכנית שומרת על חוקי המשחק, לא נותנת לי לשחק אם זה לא התור שלי, מודיעה מי ניצח ומסיימת את המשחק כשצריך, וכך הלאה.
18. התוכנית לא מאפשרת משחק אלא אם שחקן נרשם לפני כן באתר, וההרשמה הסתיימה בהצלחה.
עבור כל שחקן, התוכנית מציגה את פרטיו בתחילת המשחק, ע"י קבלתם מ- ה- *Server*.
19. כאמור קודם, כל שחקן משחק כנגד ה- *Server* הנמצא בפרויקט ה- *Razor*.
יש לשמור עבור כל משחק, מי השחקן, מועד התחלת המשחק, משך זמן המשחק, וכל פרט אחר שצריך.
ה- *Server* יגריל צעדים חוקיים, מהלך אחד בלבד בכל תור. אין לבצע עיבוד כלשהו בצד הלקוח ליצירת הצעד. אין צורך לממש אלגוריתם שמנסה לנצח.
20. כל המשחקים של כל לקוח "מוקלטים" בצד הלקוח.
לפיכך, התוכנית אצל הלקוח תאפשר שחזור משחק כלשהו שהיה לו בעבר.
במשחק משוחזר, אין צורך לשמור את משך הזמן המקורי שעבר בין תור ל- תור, כלומר כל הצעדים ישוחזרו במרווחי זמן שווים וסבירים, אולם השחזור יתמוך בשאר הדברים שהיו במשחק המקורי כגון אנימציה, צבעים, וכו'. השחזור הוא *read-only*, כלומר מאפשר רק לצפות במה שהיה.
לצורך השחזור יש לעבוד עם **<u>בסיס נתונים אחר שנמצא כאמור אצל הלקוח.</u>**
יש לאפשר בחירת משחק מתוך המשחקים שהיו. (כל לקוח שומר רק את המשחקים שלו)
ניתן לבחור לעבוד על בסיס הנתונים עם אחד המודלים הבאים:

Linq-To-Sql/ Entity Framework (EF) / ADO.NET

<u>אתר האינטרנט ושאילתות</u>

21. יש לאפשר תצוגה של תוצאות שאילתות באתר האינטרנט של המשחק.
אילוץ: עבור כל שאילתה כזו יש לממש מתודה נטולת לולאות שקודם כל תייצר את התשובה הנדרשת לשאילתה.
(כלומר, עבודה עם *Linq* כמו שעשינו בכיתה, ואין לנו *for/foreach/while* בתוך המתודה שמכינה את התוצאה.)
(אח"כ, לצורך התצוגה עצמה באתר, מותר לולאות רק אם אין אפשרות אחרת להציג את התשובה הנ"ל של השאילתה)
להלן השאילתות שהאתר יתמוך בהן:

שאילתות

22. הצגת כל השחקנים ששיחקו עד היום (יש להם לפחות משחק אחד שהסתיים) עם כל הפרטים שלהם, עם אפשרות למיון לפי השמות ללא *case sensitive*.(כלומר, כל מי ששיחק מתי שהוא, פרטיו נשמרים.)
23. הצגת כל השחקנים עם מיון בסדר יורד לפי השמות ללא *case sensitive*, כאשר יש להציג רק את השמות ובנוסף רק את התאריך של המשחק האחרון שהוא שיחק. (כלומר, שתי עמודות בלבד). (יש להציג רק שחקנים ששיחקו לפחות משחק אחד שהסתיים)
24. הצגת כל המשחקים עם כל הפרטים.
25. רוצים להציג מכל מדינה רק את השחקן הראשון ששיחק אי פעם מאותה מדינה. (יש להציג 'נציגים' אלו מכל המדינות שמהן היו אי פעם שחקנים.)
26. אני בוחר שחקן מתוך *Combo* שמציגה את השמות של כל השחקנים שיש, ללא כפילויות, בסדר עולה לפי השמות, ואז מקבל כל המשחקים שהוא שיחק. (כפילות כאן הכוונה גם אם זה אותו שם עם אותיות בגודל שונה. למשל *Avi* זה כפילות של *avI* וכך הלאה.)
27. כל שחקן, בכמה משחקים הוא שיחק. (יש להציג רק את שתי התוצאות האלו בטבלה בלי מידע נוסף)
28. טבלה או חלוקה לקבוצות, ככה שנציג שחקנים לפי כמות משחקים, עם מיון לפי הכמות בסדר יורד.
למשל, נניח שהיו שחקנים ששיחקו הכי הרבה מכולם וזה היה שלושה משחקים,
אז יש לצור טבלה באופן הבא:

שלושה משחקים:
כאן יהיו הפרטים של כל השחקנים ששיחקו שלושה משחקים.

שני משחקים:
כאן יהיו הפרטים של כל השחקנים ששיחקו רק שני משחקים.

משחק אחד:
כאן יהיו הפרטים של כל השחקנים ששיחקו רק משחק אחד.

לא שיחקו:
כאן יהיו הפרטים של כל מי שנרשם אבל לא שיחק.

29. קיבוץ של שחקנים לפי מדינות. הצגת השחקנים שיש לפי כל מדינה.
30. הצג את שתי המדינות מהן שוחקו הכי הרבה משחקים, כולל מספר משחקים. אין להציג נתונים אחרים, כלומר הטבלה שממנה תיווצר התוצאה תכיל רק שתי עמודות אלו.

עדכונים ומחיקות

בנוסף, האתר גם יאפשר עדכונים ומחיקות. יש לבצע תמיכה בשני מצבים לפי בקשת המשתמש באתר:

31. עדכון פרטים של שחקן. למשל: שם, טלפון, מדינה. (מספר מזהה - לא ניתן לשינוי/אין לאפשר)
32. מחיקה של משחק או של שחקן. אם שחקן נמחק אוטומטית נמחקים כל המשחקים שלו.
יש לבצע ברמת קשרי הגומלין של בסיס הנתונים כפי שראינו בכיתה.

(כל המחיקות יתבצעו ממש בבסיס הנתונים, ולא ע"י סימון באיזה *flag* או מנגנון דומה.)

כללי

33. פרטים רבים נותרו "פתוחים", לשיקולו האישי של הסטודנט.
אתה יכול לבצע מטלות מהנ"ל, שאין עבורן פירוט מיוחד, בכל דרך, ובלבד שתהיה התאמה סבירה וכמקובל, למטרת התוכנית.
34. יש לעבוד בפרויקט רק עם בסיסי הנתונים שהוצגו בכיתה *(Sql Server)*, עם המודלים שהוצגו בכיתה וכך הלאה.

הגשה והגנה

35. **יש להזין מראש נתונים בבסיסי הנתונים השונים, בכמות מספקת, שתאפשר לבדוק בצורה ברורה ומיידית את כל הסעיפים בעבודה זו.**
36. יש להעלות לאתר כמטלה, קובץ *zip* אחד בלבד, ובו 4 התיקיות הנדרשות. (נא להסיר את תתי התיקיות *bin, obj*.).
37. **הגשה**: בזוגות או ביחידים. (מי שעושים בזוג **רק אחד מבני הזוג** יגיש באתר.)
38. **תאריך ההגשה: עד מוצ"ש 21.03.2026, כולל.**
39. מועד ההגנה, אמור להיות בימים הקרובים למועד ההגשה ככל הניתן.
המועד יתפרסם באתר, **נא לראות את ההודעה באתר.**
40. **הגנה**: כל אחד לבד. יש להיות בקיא בכל העבודה כולה! (גם מי שעשה בזוג)
ההגנה תכלול עדכון קוד קיים, שינוי קוד, תוספת קוד וכו', כאשר עדכונים אלו אמורים לרוץ בהצלחה.
(יתכנו גם שאלות על מה שבוצע.)
41. יש לוודא מראש, שהעבודה כולה רצה בהצלחה **ובאופן מידי** על המחשב שבו אתה מציג את העבודה.

בהצלחה!